

WHICH INTERPRETABILITY DIAGNOSTICS TRACK PROGRAM CORRECTNESS? EVIDENCE FROM A COMPILED TRANSFORMER WITH FLOATING-POINT GROUND TRUTH

Anonymous authors

Paper under double-blind review

ABSTRACT

Mechanistic interpretability certifies that a network “implements an algorithm” using diagnostics computed from its behavioural surface: attention that is one-hot or low-entropy, outputs that are finite, predictions invariant to pruning, heads whose ablation is critical. Whether these track program-level correctness is an empirical question that needs ground truth of a kind trained models cannot supply. We build it. We compile the triangular solves of LU-based matrix inversion into transformer weights at binary64 precision, so that both the intended program structure *and* its floating-point semantics are known before execution, and we derive a *certified-exact* oracle for the position every attention lookup should fetch. Across 30 compiled programs and 360 execution configurations we find that diagnosticity varies by an order of magnitude and does not match how these signals are used in practice: output finiteness is close to useless (AUC 0.571), exact one-hotness and low entropy are moderate (0.826, 0.755), and invariance of the downstream prediction is the strongest (0.964). Head-ablation criticality is a poor estimator of the compiler’s own functional labels (precision 0.556, recall 0.349 against a 49% base rate). We further prove a margin bound showing that exact hard-attention positional addressing needs about $2 \log_2 P + 4.4$ significant bits to address P positions, that the attention temperature cancels from this condition, and—contrary to a natural conjecture—that bounded-magnitude re-encoding does not relax it. Our testbed is not a solver; it is an instrument, and it says which reassurance signals deserve trust.

1 INTRODUCTION

A recurring claim in mechanistic interpretability is that some network “implements” a particular algorithm. The evidence offered is usually indirect: attention distributions that are effectively one-hot, low attention entropy, outputs that remain finite and plausible, predictions that survive pruning or quantization, and ablation studies showing that particular heads are critical (Clark et al., 2019; Voita et al., 2019; Elhage et al., 2021; Wang et al., 2023). These are *proxies*. They are cheap, they are widely used, and how well they track correctness is not known, because establishing it requires a system whose intended computation is known independently of the network’s behaviour.

Compiled transformers supply part of that ground truth. Tracr (Lindner et al., 2023) compiles RASP programs (Weiss et al., 2021) into weights; ALTA (Shaw et al., 2025) and looped constructions (Giannou et al., 2023) extend the compilation target; InterpBench (Gupta et al., 2024) and MIB (Mueller et al., 2025) use known-circuit models to score interpretability methods. What none of them provides is a *numerical* axis. Their programs are symbolic, so there is no notion of the computation being carried out to a stated accuracy, and no way to perturb precision and ask whether the program is still computing the right values.

We add that axis. CRAFT compiles the forward and backward substitution of LU-based matrix inversion into transformer weights at binary64, for symmetric positive-definite Laplacian systems.

Because the program is emitted rather than learned, three things are known before any execution: which dimension holds which intermediate quantity, which attention head is supposed to fetch from which position, and what the exact arithmetic result should be. Perturbing the execution—dtype, quantization, pruning, head ablation—then moves the program away from correctness in a measurable way, while the behavioural proxies can be read off exactly as a practitioner would read them.

This yields a question that can actually be answered:

When a numerical program is compiled into transformer weights with ground-truth operation labels available, which of the standard diagnostics used to certify that a transformer “implements an algorithm” actually track program-level correctness under precision and quantization perturbation, and which give false reassurance?

Our contributions:

1. **A ground-truth testbed with a certified-exact oracle** (§3). We derive, for every attention lookup, the position the program must fetch, certified by a rounding-error bound with exact rational fallback. Over 104,381 lookup decisions, 98.1% are certified directly and the rest are resolved exactly. The oracle is a property of the compiled program, not of any particular run, which is what lets us score a perturbed execution without circularity.
2. **A measurement of proxy diagnosticity** (§5). Across 30 programs $\times$ 12 configurations we report, for each proxy, its green rate on correct runs, its false-reassurance rate, a likelihood ratio, and the AUC of its continuous value against compiler-labelled correctness. The spread is large and the ranking is not the one practice assumes.
3. **A head-ablation audit against compiler labels** (§6). Ablation criticality recovers the compiler’s own lookup/passthrough labels with precision 0.556 and recall 0.349.
4. **A precision requirement for compiled attention addressing** (§7). Exact selection up to position P needs roughly $2 \log_2 P + 4.4$ significand bits. The attention temperature K cancels entirely from the precision condition and enters only a dynamic-range condition, so the large K used in constructive results buys nothing and costs range. We also show that normalizing the positional encoding does *not* help, because it shrinks the margin exactly as fast as the magnitudes.
5. **Evidence that the compiled program executes run-time inputs** (§4), so the proxy results are about a solver and not about a replayed constant.

We are explicit that CRAFT is not a useful solver—it is about five orders of magnitude slower than a library call—and that its compiler accepts only coefficients known at build time. It is an instrument. Its value is that every number it produces can be checked.

2 RELATED WORK

Compiled and constructive transformers. RASP (Weiss et al., 2021) and Tracr (Lindner et al., 2023) compile symbolic programs into transformer weights; ALTA (Shaw et al., 2025) targets a lower-level assembly; Giannou et al. (2023) build programmable looped transformers. Expressivity results establish what hard-attention transformers can represent (Pérez et al., 2019; 2021; Hahn, 2020; Barceló et al., 2024; Merrill & Sabharwal, 2023b;a), surveyed by Strobl et al. (2024). These are stated over the reals or with idealized precision; our contribution is to ask what precision the constructions actually require.

Interpretability evaluation with known ground truth. InterpBench (Gupta et al., 2024) and MIB (Mueller et al., 2025) evaluate circuit discovery against known structure; ACDC (Conmy et al., 2023) and causal scrubbing (Chan et al., 2022) are the methods being evaluated. Miller et al. (2024) show circuit-faithfulness metrics are sensitive to methodological choices. Our axis is complementary and, to our knowledge, absent: these benchmarks have no notion of a program being numerically wrong.

Architecture. A controller that mutates an external memory between steps is a Neural Turing Machine / DNC (Graves et al., 2014; 2016); our runner-plus-cache is that design. Our attention

lookup is 2-D maximum inner product search, i.e. the dynamic convex hull trick (Overmars & van Leeuwen, 1981; Brodal & Jacob, 2002; Li, 2026); attention-as-MIPS is also the basis of LSH attention (Shrivastava & Li, 2014; Kitaev et al., 2020).

Numerical analysis. The backward error we measure is the classical normwise quantity of Rigal & Gaches (1967), with Oettli & Prager (1964) as the componentwise ancestor; see Higham (2002, Thm 7.1). LU stability and growth factors are classical (Wilkinson, 1961; 1963; Higham, 2002). Our η measurements confirm this theory rather than extending it; we report them as calibration. Verified numerics (Rump, 1983; 1999; 2010; Ogita et al., 2005) and machine-checked floating-point proofs (Boldo & Melquiond, 2011; Daumas & Melquiond, 2010; Roux, 2016; Kellison et al., 2023; Kwan & Hunt Jr., 2024) give stronger guarantees for linear solvers than an a-posteriori certificate; our claim is not to compete with them but to apply their standard of evidence inside a transformer.

Quantization and pruning. We use per-tensor and per-channel int8 (Jacob et al., 2018; Krishnamoorthi, 2018; Nagel et al., 2021; Dettmers et al., 2022), bfloat16 (Kalamkar et al., 2019; Burgess et al., 2019), and magnitude pruning (Han et al., 2015; 2016; Frankle & Carbin, 2019).

3 THE TESTBED AND ITS ORACLE

What is compiled. The host computes the Doolittle factors $\hat{L}, \hat{U}$ of A_{FF} once at build time and bakes them into token embeddings. The transformer executes forward and backward substitution token by token. A runner threads values between steps through the KV cache, supplying b_i at the rhs_i token and y_i at bck_i . We do not claim the factorization is executed by the network.

Scope. The compiler accepts only coefficients known at build time. No compiled program here can express pivoting, input-dependent control flow, or input-dependent coefficients, and each matrix is compiled separately. This is a ceiling on the framework, and it bounds what “a transformer executes a numerical routine” can mean in this setting.

Two populations. Of 154 circuits in our suite, the target node is a *fixed* source node in 12. For those the compiler emits a direct source read: the substitution machinery is compiled and scheduled but never exercised, and 88 free-variable solve steps are built and never run. Those 12 attain zero downstream error without executing a solve. We report every execution result on the 142 that do execute, and we flag this because it is exactly the failure mode a testbed should catch: a program can pass an application-level metric without running.

The oracle. Let $s(p) = \langle k_p, q_t \rangle$ be the attention score at position p . The position the program *must* fetch is $\arg \max_p s(p)$ evaluated exactly. Computing that in rational arithmetic at every lookup is expensive, so we certify instead: if the float64 winning margin exceeds a bound on the accumulated rounding error of a two-term dot product, the computed $\arg \max$ is provably the exact one; otherwise we recompute that row with `Fraction`. Over 104,381 lookup decisions on compiler-labelled lookup heads, 98.1% were certified directly (minimum 95.7% per program) and 927 required the exact fallback.

This matters for validity. Defining correctness as “selects what the float64 run selected” would be circular for the precision arm, since any dtype change would then be incorrect by construction. Our oracle is a property of the compiled program, so a perturbed configuration is scored against what the program *means*.

Head labels. The compiler assigns each attention head a role: *lookup* heads implement a scheduled fetch, *passthrough* heads carry slot contents forward using an erase query. We read these labels from the compiler’s allocation record. This distinction is load-bearing: pooling the two makes exact one-hotness unattainable even on provably correct runs (it caps at 0.22), which would make the proxy unmeasurable. All proxies below are computed on lookup heads.

Table 1: Proxy diagnosticity against the certified-exact compiler oracle. 30 programs, 360 configurations, 210 broken. $LR_{\text{green}} = P(\text{green} \mid \text{correct})/P(\text{green} \mid \text{broken})$; 1.0 is uninformative. AUC is rank discriminability of the proxy’s continuous value; 0.5 is chance. Intervals are cluster bootstraps over programs.

Proxy	$P(\text{green} \mid \text{correct})$	$P(\text{green} \mid \text{broken})$	LR_{green}	AUC
outputs all finite	1.000	0.857	1.17	0.571
attention entropy ≈ 0	1.000	0.490	2.04	0.755
softmax exactly one-hot	1.000	0.348	2.88	0.826
prediction unchanged	1.000	0.071	14.00	0.964
all four simultaneously	—	0.019	—	—

4 THE COMPILED PROGRAM EXECUTES RUN-TIME INPUTS

If the compiled model only ever ran on one input, its attention pattern would say nothing about computation. We therefore drive each program with six families of right-hand side, **with no re-compilation between them**: columns of the identity, the physical source-driven b , i.i.d. Gaussian, wide-dynamic-range vectors spanning 10^{-8} to 10^8 , the all-ones forcing vector solved in-model, and a vector aligned with the smallest singular direction of A .

Over 40 matrices and 1,163 solves, the maximum per-column normwise backward error is $1.88 \times 10^{-16} \approx 1.7u$ and the selector hit rate against the oracle is exactly 1.000 in every family; median η ranges from 1.19×10^{-17} to 4.16×10^{-17} with no family separating from the others (Appendix B). The program computes a genuine function of its input, at the accuracy classical theory predicts for a triangular solve with a growth factor near 1 (Higham, 2002).

5 MAIN RESULT: DIAGNOSTICITY VARIES BY AN ORDER OF MAGNITUDE

Design. 30 compiled programs ($n \in [3, 44]$ unknowns) $\times$ 12 execution configurations: dtype {f64, f32, bf16, f16}; quantization {per-tensor int8, per-channel int8}; and magnitude pruning at {10, 50, 90}% computed both over all coefficients and over *nonzero* coefficients only. 360 independent cells. A run is *broken* when any lookup fetches a position other than the oracle’s, or when the output is not finite.

Proxies. (i) all outputs finite; (ii) softmax exactly one-hot; (iii) attention entropy ≈ 0 , computed as true Shannon entropy with no regulariser; (iv) downstream prediction unchanged from the f64 run.

Table 1 is the paper’s main result, and it is more discriminating than a blanket “proxies fail”. Every proxy is green on every correct run, so none produces false alarms. They differ sharply in what a green reading is worth:

- **Output finiteness is close to useless** (AUC 0.571, LR 1.17). It stays green in 85.7% of runs in which the program provably fetches wrong values. It is also the cheapest and most commonly applied sanity check.
- **Exact one-hotness and near-zero entropy are moderate** (0.826, 0.755). Notably, these are only measurable at all once heads are separated by compiler role; pooled over all heads they are degenerate.
- **Prediction invariance is the strongest signal** (AUC 0.964, LR 14.0), despite being measured on a coarse 101-point output grid. This is the opposite of what a mechanistic account would suggest—an end-to-end behavioural check outperforms three internal, mechanism-flavoured ones.

Only 1.9% of broken runs are green on all four simultaneously, so a *conjunction* of diagnostics is far safer than any single one. That is an actionable recommendation, and it is not what single-proxy practice does.

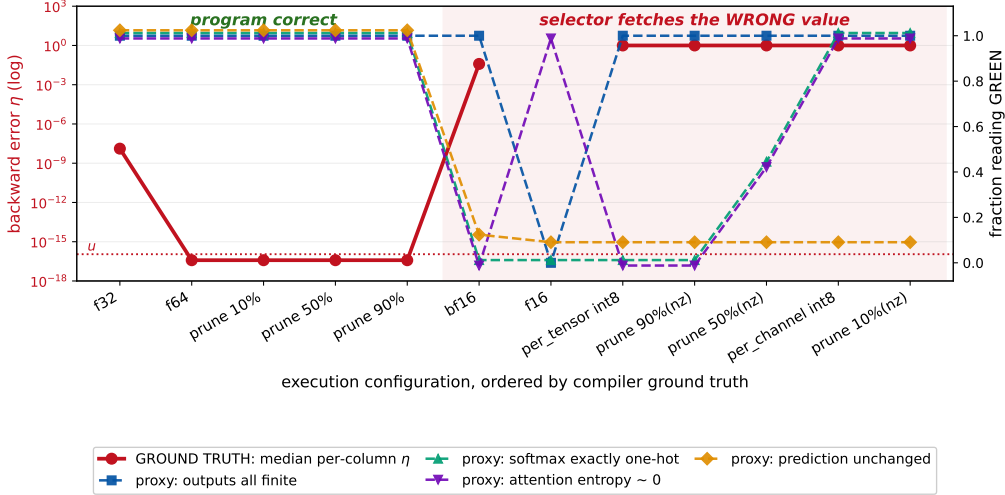


Figure 1: Ground truth (red, left log axis) versus the four proxies (right axis) across execution configurations ordered by compiler-labelled correctness. In the shaded region the selector probably fetches wrong values while proxy curves remain high—most conspicuously output finiteness.

Table 2: Head ablation versus the compiler’s own functional labels.

Compiler label	heads	ablation changes prediction	ablation breaks program
lookup	86	30 (34.9%)	47 (54.7%)
passthrough	88	24 (27.3%)	61 (69.3%)

Pruning invariance is structural sparsity. Pruning 10/50/90% of *all* coefficients leaves execution bit-identical ($\eta = 3.9 \times 10^{-17}$, hit rate 1.000). Pruning 10% of the *nonzero* coefficients destroys it ($\eta = 1.0$, hit rate 0.130). Reports of extreme pruning invariance in compiled models can therefore be measuring how sparse the emitted tensor already is. The distinction costs one line of code and reverses the conclusion.

The attention temperature is not the mechanism. A natural explanation for quantization collapse is that a large attention scale K dominates the tensor range and rounds the arithmetic coefficients away. We tested it by recompiling at $K \in \{10^{10}, 10^8, 10^6, 10^4\}$: the int8 hit rate moves from 0.898 to 0.903 and η stays at 1.00 throughout. Six orders of magnitude of K change nothing, consistent with §7 where K cancels analytically. Per-channel int8 (Dettmers et al., 2022) does not rescue execution either.

6 HEAD-ABLATION CRITICALITY AGAINST COMPILER LABELS

Ablation is the standard tool for attributing function to heads. With compiler labels we can score it directly. We zero each head’s output contribution in turn across 8 programs (174 labelled heads) and ask whether the prediction changes.

Treating “ablation changes the prediction” as a detector for “the compiler assigned this head a lookup role” gives precision 0.556 and recall 0.349, against a base rate of $86/174 = 49.4\%$ lookup heads. Ablation criticality is close to uninformative about compiler-assigned role here: it misses two thirds of the heads that genuinely perform scheduled fetches, and nearly half of the heads it flags are passthrough. Two mechanisms explain this without any appeal to redundancy: many lookup heads feed quantities the coarse output grid cannot resolve, and passthrough heads are load-bearing for state propagation even though they compute nothing.

Table 3: Predicted ceilings at the production scale $K = 10^{10}$ against measured first-failure positions. The bound is sufficient, hence conservative where precision binds; for float16 the *range* condition binds and predicts immediate failure, which is what occurs.

Format	$P_{\text{precision}}$	P_{range}	binding	predicted P	measured first failure
float64	2.06×10^7	7.7×10^{148}	precision	2.06×10^7	none up to 4×10^6
float32	891	1.1×10^{14}	precision	891	3898
bfloat16	4.92	1.1×10^{14}	precision	4.92	15
float16	9.84	0.0015	range	0.0015	1

7 HOW MUCH PRECISION DOES COMPILED ADDRESSING NEED?

Constructive transformer results are stated over the reals. The following says what they cost in bits. The compiler scales the query by K , so with keys $k_p = (2p, -p^2 + \alpha g(p))$ and query $q_t = K'(t, 1)$, $K' = K\sqrt{d_h}$,

$$s(p) = K'(2pt - p^2 + \alpha g(p)) = K'(t^2 - (t - p)^2 + \alpha g(p)),$$

and the t^2 term cancels from every pairwise comparison.

Lemma 1 (Selector exactness). *Let $0 \leq g < 1/\ln 2$ and $0 < \alpha < \ln 2$, and write $M = 1 - \alpha/\ln 2$. For $p \neq t$, $s(t) - s(p) \geq K'M$. If all positions satisfy $p, t \leq P$ with $P \geq 1$, then the computed $\arg \max$ in a binary floating-point format with unit roundoff u and largest finite value Ω equals the exact $\arg \max$ whenever*

$$(\text{precision}) \quad P < \sqrt{\frac{M}{12u}}, \quad \text{and} \quad (\text{range}) \quad P < \sqrt{\frac{\Omega}{3K'}}.$$

The scale K' cancels from the precision condition and appears only in the range condition.

A proof is in Appendix A. Equivalently, addressing P positions exactly requires about $2 \log_2 P + \log_2(12/M) \approx 2 \log_2 P + 4.4$ significant bits: 11.0 bits for $P = 10$, 22.9 for $P = 601$, 31.0 for $P = 10^4$.

Bounded-magnitude re-encoding does not help. It is tempting to blame low-precision failure on unbounded score magnitude and propose normalizing positions, $p \mapsto p/P$. This does not work, and the reason generalizes: normalization divides the margin by P^2 exactly as it divides the magnitudes, leaving the relative gap unchanged, so the condition becomes $P < 1/\sqrt{2u}$ — the same $O(u^{-1/2})$ scaling. Empirically, the normalized encoding first fails at position 15 in bfloat16, identical to the production encoding. The requirement is intrinsic to quadratic positional keys, not an artifact of a badly chosen constant.

Consequence. The large attention temperature used to force hard attention is a pure liability: it cannot buy exactness (Lemma 1) and it shrinks the representable position range. Compiled-transformer constructions that assume real arithmetic should state a precision budget, and this lemma gives it.

Table 4 gives the ground truth per configuration. Two features are worth noting. First, the correct/broken boundary is sharp: every configuration is either entirely correct across all 30 programs or entirely broken, which is what one expects when correctness is governed by a margin condition rather than by accumulated noise. Second, the *severity* ordering is not the intuitive one—per-channel int8 degrades the selector further than per-tensor, and nonzero-pruning at 10% is the most destructive configuration tested, worse than dropping to float16.

8 DISCUSSION

Our results suggest three concrete changes to how compiled-model evidence is reported.

Table 4: Ground truth by execution configuration (30 programs each). A configuration is broken when any lookup deviates from the oracle. Note the separation between pruning computed over all coefficients and over nonzero coefficients only, and that per-channel int8 is *worse* here than per-tensor.

Config	frac. broken	med. hit	med. η	Config	frac. broken	med. hit	med. η
f64	0.00	1.000	3.9×10^{-17}	bf16	1.00	0.915	3.9×10^{-2}
f32	0.00	1.000	1.3×10^{-8}	f16	1.00	0.628	∞
prune 10%	0.00	1.000	3.9×10^{-17}	per-tensor int8	1.00	0.628	1.00
prune 50%	0.00	1.000	3.9×10^{-17}	per-channel int8	1.00	0.241	1.00
prune 90%	0.00	1.000	3.9×10^{-17}	prune 10%(nz)	1.00	0.130	1.00

Report a conjunction, not a single proxy. No single diagnostic is safe: the best one still reads green in 7% of broken runs, and the most commonly used one in 86%. All four together fail only 1.9% of the time. Since the proxies are cheap and largely independent, reporting them jointly is close to free.

Separate heads by role before measuring internal proxies. One-hotness and entropy are unmeasurable when lookup and passthrough heads are pooled—the pooled statistic never reaches the value a correct run attains. Any claim of the form “attention is one-hot, so the model is doing lookup” needs the head partition to be stated. In a compiled model it is available; in a trained model it is precisely what is being inferred, which is a reason for caution.

State a precision budget. Constructive transformer results assume real arithmetic. Lemma 1 converts that assumption into a number of significant bits, and shows that the standard trick of raising the attention temperature does not buy accuracy—it only costs dynamic range. A construction that addresses 10^4 positions needs 31 bits of significand and therefore cannot be executed in any 16-bit format, however it is scaled.

Finally, a methodological point. Two results here were only visible because the ground truth was a *compiler* rather than a reference run: that 12 of 154 programs pass the application metric without executing, and that pruning invariance was a statement about tensor sparsity. Both would have been reported as successes by a pipeline that compared against its own float64 trace.

9 LIMITATIONS

Compiled, not trained. Our object of study is a hand-compiled, untrained, structurally sparse network. We show which proxies track correctness *in a setting where correctness is checkable*; whether the same ranking holds for gradient-trained models is not established here, and is the main threat to external validity. We think the finiteness result is likeliest to transfer (it is a property of the metric, not of the network) and the ablation result least.

Perturbations are strong. int8, bfloat16 and nonzero-pruning are aggressive. We do not claim practitioners believe these proxies certify numerical correctness under quantization; we claim that when they *are* read as health signals, their diagnosticity differs by an order of magnitude, and that ordering was not previously measurable.

Scale and scope. $n \leq 44$ unknowns, one compiler, one architecture family, one program family (triangular substitution), build-time-constant coefficients only, no pivoting. Ablation covers 174 labelled heads on 8 programs.

Reduction order depends on a MILP solve. Compiling one circuit under five solver configurations yields five distinct weight files; for one of four circuits tested the *executed result* changed by 1.84 ulp, traceable to a different slot assignment and hence a different FFN reduction order. Bitwise reproducibility claims about compiled models must be scoped to a fixed schedule (Appendix C).

Not a solver. About $10^5 \times$ slower than a library solve.

REPRODUCIBILITY STATEMENT

All results come from scripts released with the submission, run on CPU in float64 under a single pinned source tree. Appendix A contains the proof of Lemma 1; Appendix B the full run-time right-hand-side table; Appendix C the schedule-stability data; Appendix E the environment, per-script runtimes and entry points. The oracle’s certification rate and exact-fallback counts are reported per program so the ground truth itself can be audited.

REFERENCES

- Pablo Barceló, Alexander Kozachinskiy, Anthony Widjaja Lin, and Vladimir Podolskii. Logical languages accepted by transformer encoders with hard attention. In *The Twelfth International Conference on Learning Representations (ICLR)*, pp. 22077–22087, 2024. URL <https://openreview.net/forum?id=gbrHZq07mq>.
- Sylvie Boldo and Guillaume Melquiond. Flocq: A unified library for proving floating-point algorithms in Coq. In *2011 IEEE 20th Symposium on Computer Arithmetic (ARITH)*, pp. 243–252, Tübingen, Germany, July 2011. IEEE. doi: 10.1109/ARITH.2011.40.
- Gerth Stølting Brodal and Riko Jacob. Dynamic planar convex hull. In *Proceedings of the 43rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 617–626. IEEE Computer Society, 2002. doi: 10.1109/SFCS.2002.1181985. Full version: arXiv:1902.11169.
- Neil Burgess, Jelena Milanovic, Nigel Stephens, Konstantinos Monachopoulos, and David Mansell. Bfloat16 processing for neural networks. In *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*, pp. 88–91, 2019. doi: 10.1109/ARITH.2019.00022.
- Lawrence Chan, Adrià Garriga-Alonso, Nicholas Goldowsky-Dill, Ryan Greenblatt, Jenny Nitishinskaya, Ansh Radhakrishnan, Buck Shlegeris, and Nate Thomas. Causal Scrubbing: a method for rigorously testing interpretability hypotheses. AI Alignment Forum, December 2022. URL <https://www.alignmentforum.org/posts/JvZhhzycHu2Yd57RN/causal-scrubbing-a-method-for-rigorously-testing>. Redwood Research technical report; not peer-reviewed. Posted 3 December 2022; cross-posted to LessWrong. Authors listed alphabetically. <https://www.alignmentforum.org/posts/JvZhhzycHu2Yd57RN/causal-scrubbing-a-method-for-rigorously-testing>.
- Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does BERT look at? an analysis of BERT’s attention. In Tal Linzen, Grzegorz Chrupala, Yonatan Belinkov, and Dieuwke Hupkes (eds.), *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 276–286, Florence, Italy, August 2019. Association for Computational Linguistics. doi: 10.18653/v1/W19-4828. URL <https://aclanthology.org/W19-4828/>.
- Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, volume 36, pp. 16318–16352. Curran Associates, Inc., 2023. doi: 10.52202/075280-0719. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/34e1dbe95d34d7ebaf99b9bcaeb5b2be-Paper-Conference.pdf. NeurIPS 2023 Spotlight.
- Marc Daumas and Guillaume Melquiond. Certification of bounds on expressions involving rounded operators. *ACM Transactions on Mathematical Software*, 37(1):2:1–2:20, January 2010. doi: 10.1145/1644001.1644003. Gappa is available at <https://gappa.gitlabpages.inria.fr/>.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In S. Koyejo, S. Mohamed,

- A. Agarwal, D. Belgrave, K. Cho, and A. Oh (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 30318–30332. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/c3ba4962c05c49636d4c6206a97e9c8a-Abstract-Conference.html.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>. Published 22 December 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *7th International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, May 2019. OpenReview.net. URL <https://openreview.net/forum?id=rJl-b3RcF7>.
- Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 11398–11442. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/giannou23a.html>.
- Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. URL <https://arxiv.org/abs/1410.5401>.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626): 471–476, 2016. doi: 10.1038/nature20101.
- Rohan Gupta, Iván Arcuschin, Thomas Kwa, and Adrià Garriga-Alonso. Interpbench: Semi-synthetic transformers for evaluating mechanistic interpretability techniques. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track*, volume 37, pp. 92922–92951. Curran Associates, Inc., 2024. doi: 10.52202/079017-2950. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/a8f7d43ae092d9a5295775eb17f3f4f7-Paper-Datasets_and_Benchmarks_Track.pdf.
- Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 2020. doi: 10.1162/tacl.a.00306. URL <https://aclanthology.org/2020.tacl-1.11/>.
- Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural network. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, volume 28, pp. 1135–1143. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/ae0eb3eed39d2bcef4622b2499a05fe6-Abstract.html.
- Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016. doi: 10.48550/arXiv.1510.00149. URL <https://arxiv.org/abs/1510.00149>. Oral presentation; Best Paper Award.
- Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Number 80 in Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, second edition, 2002. ISBN 0-89871-521-0. doi: 10.1137/1.9780898718027.

- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.
- Dhiraj Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. A study of BFLOAT16 for deep learning training. *arXiv preprint arXiv:1905.12322*, 2019. URL <https://arxiv.org/abs/1905.12322>.
- Ariel E. Kellison, Andrew W. Appel, Mohit Tekriwal, and David Bindel. LAProof: A library of formal proofs of accuracy and correctness for linear algebra programs. In *2023 IEEE 30th Symposium on Computer Arithmetic (ARITH)*, pp. 36–43. IEEE, September 2023. doi: 10.1109/ARITH58626.2023.00021.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=rkgNKkHtvB>. arXiv:2001.04451.
- Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. URL <https://arxiv.org/abs/1806.08342>.
- Carl Kwan and Warren A. Hunt Jr. Formalizing the Cholesky factorization theorem. In Yves Bertot, Temur Kutsia, and Michael Norrish (eds.), *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pp. 25:1–25:16, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-337-9. doi: 10.4230/LIPIcs.ITP.2024.25.
- Chao Li. The li-chao tree: Algorithm specification and analysis, 2026. URL <https://arxiv.org/abs/2603.07948>.
- David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Tom McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, volume 36, pp. 37876–37899. Curran Associates, Inc., 2023. doi: 10.52202/075280-1649. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/771155abaae744e08576f1f3b4b7ac0d-Paper-Conference.pdf.
- William Merrill and Ashish Sabharwal. A logic for expressing log-precision transformers. In *Advances in Neural Information Processing Systems*, volume 36, pp. 52453–52463. Curran Associates, Inc., 2023a. doi: 10.52202/075280-2284. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/a48e5877c7bf86a513950ab23b360498-Abstract-Conference.html.
- William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023b. doi: 10.1162/tacl.a.00562. URL <https://aclanthology.org/2023.tacl-1.31/>.
- Joseph Miller, Bilal Chughtai, and William Saunders. Transformer circuit faithfulness metrics are not robust. In *First Conference on Language Modeling (COLM)*, 2024. URL <https://arxiv.org/abs/2407.08734>.
- Aaron Mueller, Atticus Geiger, Sarah Wiegrefe, Dana Arad, Iván Arcuschin, Adam Belfki, Yik Siu Chan, Jaden Fried Fiotto-Kaufman, Tal Haklay, Michael Hanna, Jing Huang, Rohan Gupta, Yaniv Nikankin, Hadas Orgad, Nikhil Prakash, Anja Reusch, Aruna Sankaranarayanan, Shun Shao, Alessandro Stolfo, Martin Tutek, Amir Zur, David Bau, and Yonatan Belinkov. MIB: A mechanistic interpretability benchmark. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd*

- International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 45069–45108. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/mueller25a.html>.
- Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021. URL <https://arxiv.org/abs/2106.08295>.
- W. Oettli and W. Prager. Compatibility of approximate solution of linear equations with given error bounds for coefficients and right-hand sides. *Numerische Mathematik*, 6(1):405–409, 1964. doi: 10.1007/BF01386090.
- Takeshi Ogita, Siegfried M. Rump, and Shin’ichi Oishi. Accurate sum and dot product. *SIAM Journal on Scientific Computing*, 26(6):1955–1988, 2005. ISSN 1064-8275. doi: 10.1137/030601818.
- Mark H. Overmars and Jan van Leeuwen. Maintenance of Configurations in the Plane. *Journal of Computer and System Sciences*, 23(2):166–204, 1981. doi: 10.1016/0022-0000(81)90012-X.
- Jorge Pérez, Javier Marinković, and Pablo Barceló. On the Turing completeness of modern neural network architectures. In *International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, 2019. URL <https://openreview.net/forum?id=HyGBdo0qFm>.
- Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is Turing-complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021. URL <https://jmlr.org/papers/v22/20–302.html>.
- J. L. Rigal and J. Gaches. On the compatibility of a given solution with the data of a linear system. *Journal of the ACM*, 14(3):543–548, July 1967. doi: 10.1145/321406.321416.
- Pierre Roux. Formal proofs of rounding error bounds: With application to an automatic positive definiteness check. *Journal of Automated Reasoning*, 57(2):135–156, August 2016. ISSN 0168-7433. doi: 10.1007/s10817-015-9339-z.
- Siegfried M. Rump. Solving algebraic problems with high accuracy. In Ulrich W. Kulisch and Willard L. Miranker (eds.), *A New Approach to Scientific Computation*, pp. 51–120. Academic Press, New York, 1983. ISBN 978-0-12-428660-3. doi: 10.1016/B978-0-12-428660-3.50010-0.
- Siegfried M. Rump. INTLAB — INTerval LABoratory. In Tibor Csendes (ed.), *Developments in Reliable Computing*, pp. 77–104. Kluwer Academic Publishers, Dordrecht, 1999. ISBN 978-90-481-5350-3. doi: 10.1007/978-94-017-1247-7_7. Software available at <https://www.tuhh.de/ti3/rump/intlab/>.
- Siegfried M. Rump. Verification methods: Rigorous results using floating-point arithmetic. *Acta Numerica*, 19:287–449, May 2010. ISSN 0962-4929. doi: 10.1017/S096249291000005X.
- Peter Shaw, James Cohan, Jacob Eisenstein, Kenton Lee, Jonathan Berant, and Kristina Toutanova. ALTA: Compiler-based analysis of transformers. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=h751wl9xiR>.
- Anshumali Shrivastava and Ping Li. Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips). In Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger (eds.), *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, volume 27, pp. 2321–2329. Curran Associates, Inc., 2014. URL https://proceedings.neurips.cc/paper_files/paper/2014/hash/c98e7c4b8f20d384e3ad857d0ee226cc-Abstract.html. arXiv:1405.5869.
- Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 2024. doi: 10.1162/tacl_a-00663. URL <https://aclanthology.org/2024.tacl-1.30/>.

Elena Voita, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. In Anna Korhonen, David Traum, and Lluís Màrquez (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 5797–5808, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1580. URL <https://aclanthology.org/P19-1580/>.

Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, Kigali, Rwanda, May 1–5, 2023. OpenReview.net, 2023. URL <https://openreview.net/forum?id=NpsVSN6o4u1>.

Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 11080–11090. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/weiss21a.html>.

J. H. Wilkinson. Error analysis of direct methods of matrix inversion. *Journal of the ACM*, 8(3): 281–330, July 1961. doi: 10.1145/321075.321076.

J. H. Wilkinson. *Rounding Errors in Algebraic Processes*. Prentice-Hall, Englewood Cliffs, NJ, USA, 1963. Also published as Notes on Applied Science No. 32, Her Majesty’s Stationery Office, London; reprinted by Dover, New York, 1994 (ISBN 0-486-67999-3) and by SIAM, Philadelphia, 2023, Classics in Applied Mathematics (ISBN 978-1-61197-751-6).

A PROOF OF LEMMA 1

Exact margin. For $p \neq t$, $s(t) - s(p) = K' [(t - p)^2 + \alpha(g(t) - g(p))]$. Since $p \neq t$ are integers, $(t - p)^2 \geq 1$; since $0 \leq g < 1/\ln 2$, $|g(t) - g(p)| < 1/\ln 2$. Hence $s(t) - s(p) \geq K'(1 - \alpha/\ln 2) = K'M > 0$ for $\alpha < \ln 2$. Numerically, with $\alpha = 0.3$ we have $M = 0.5672$, and the minimum exact gap measured over $t \in [1, 3 \times 10^5]$ is 0.9996, so the bound holds with room.

Rounding. Each score is a two-term dot product. Under the standard model $\text{fl}(a \text{ op } b) = (a \text{ op } b)(1 + \delta)$, $|\delta| \leq u$, two products and one addition give $|\text{fl}(s(p)) - s(p)| \leq \gamma_2 S_p$ with $\gamma_2 = 2u/(1 - 2u)$ and $S_p = K'(|2pt| + | - p^2 + \alpha g(p)|)$, the absolute-value sum that bounds cancellation. For $p, t \leq P$ and $P \geq 1$, $S_p \leq 3K'P^2$.

Precision condition. The computed $\arg \max$ is exact if the exact margin exceeds the worst combined perturbation of the two scores compared: $K'M > \gamma_2(S_t + S_p)$, which is implied by $M > 6\gamma_2 P^2$. Using $\gamma_2 \leq 2u/(1 - 2u)$ and $u \leq \frac{1}{2}$, a sufficient condition is $M > 12uP^2$, i.e. $P < \sqrt{M/(12u)}$. The factor K' appears on both sides and cancels. $\square$

Range condition. Representability requires $\max_p |s(p)| \leq 3K'P^2 \leq \Omega$, giving $P < \sqrt{\Omega/(3K')}$, which does depend on K' . The valid range is the minimum of the two conditions.

B RUN-TIME RIGHT-HAND SIDES

C SCHEDULE STABILITY

Each circuit was compiled under five MILP configurations (three solver seeds, a reduced time limit, and a different solver backend) and executed on identical inputs. All five produce distinct weight files. For three of four circuits the executed result is bit-identical; for one it differs by 1.84 ulp, tracking a third distinct slot map. The emitted program is not byte-stable across solver configurations, and the executed result is not always bit-stable either.

Table 5: 40 matrices, 1,163 solves, no recompilation between right-hand sides. $u = 1.11 \times 10^{-16}$. Selector hit rate is against the certified-exact oracle.

RHS family	solves	$\max \eta$	median η	min selector hit
identity columns e_j	243	1.58×10^{-16}	1.25×10^{-17}	1.000
source-driven b	40	1.88×10^{-16}	4.16×10^{-17}	1.000
i.i.d. Gaussian	400	1.64×10^{-16}	1.44×10^{-17}	1.000
wide dynamic range	400	1.78×10^{-16}	2.11×10^{-17}	1.000
all-ones, in-model	40	1.09×10^{-16}	1.36×10^{-17}	1.000
adversarial	40	1.18×10^{-16}	1.19×10^{-17}	1.000

D DOWNSTREAM METRIC DECOMPOSITION

The application-level readout quantizes to a 50 mV grid. Decomposing the error over all 154 circuits: total 11.84 mV mean; readout quantization 11.84 mV; target-model mismatch 0.025 mV; and the solve itself 1.6×10^{-10} mV. Total and quantization error agree to four significant figures, and the uniform-in-bin expectation is 12.5 mV. The metric measures the readout budget, not solver accuracy; we use it in §5 only as a behavioural proxy, which is the one role it can support. We also report that 0 of 1437 solution entries are negative, so the pipeline’s non-negativity clamp is a no-op on the declared domain.

E REPRODUCIBILITY DETAILS

CPU only; float64 throughout (the attention scale overflows in float32, and the workload is dispatch-bound rather than FLOP-bound, so no GPU is used or needed). Compiled models are regenerated deterministically from the build scripts given a fixed MILP configuration; see Appendix C for the caveat. Each program builds in 1–45 s and solves in under a second. Entry points, exact package versions and per-script runtimes are in the released README.

F USE OF LARGE LANGUAGE MODELS

An LLM-based coding assistant was used substantially in this work and we disclose that here in line with the conference policy. Its role was: implementing the experiment scripts and the certified-exact oracle; running the experiments and tabulating results; drafting and revising this manuscript; and conducting the literature search, including retrieving and cross-checking bibliographic records. Every citation in this paper was independently verified to exist against a primary source (publisher DOI records, ACL Anthology, or proceedings pages) before inclusion, and one candidate reference that could not be verified was removed. The research questions, the experimental designs, the interpretation of results and all scientific claims are the authors’ responsibility, and the authors have checked the derivations and re-run the reported experiments.